

UNIVERSIDAD DEL VALLE DE GUATEMALA

PRUEBAS AUTOMATIZADAS

Informe técnico de implementación y resultados

Sistema de cuidado del adulto mayor

Curso	Entrega	Equipo
CC3091 - Ingeniería de Software 2	Tarea 3 - Semestre II 2026	[Agregar nombres y carnés]

Guatemala, julio de 2026

Resumen ejecutivo

Se investigaron alternativas para automatizar pruebas en el frontend JavaScript y el backend PHP/Laravel del sistema de cuidado del adulto mayor. Se seleccionaron Vitest con jsdom y Testing Library para el frontend, y PHPUnit integrado con Laravel para el backend. La implementación verificada comprende 102 pruebas: 14 de frontend y 88 de backend, todas aprobadas.

Capa	Herramientas	Archivos	Pruebas	Resultado	Tiempo observado
Frontend	Vitest 3.2.7 + jsdom + Testing Library	4	14	14 aprobadas	1.45 s
Backend	PHPUnit 11.5 + Laravel Test	17	88 (405 aserciones)	88 aprobadas	1.67 s
Total	Dos suites automatizadas	21	102	0 fallos	3.12 s

Resultado principal

Las dos suites finalizaron sin errores. Los tiempos corresponden a las ejecuciones realizadas el 19 de julio de 2026 en el equipo local y pueden variar según el entorno.

1. Objetivo y alcance

El objetivo fue seleccionar herramientas apropiadas para la tecnología del proyecto, implementar una muestra representativa de pruebas automatizadas y evaluar su utilidad. El alcance incluye lógica unitaria, manipulación del DOM, comunicación con la API, autenticación, autorización, validaciones, persistencia y reglas de negocio.

Competencias cubiertas

- Investigación de frameworks para JavaScript y PHP/Laravel.
- Selección razonada de Vitest y PHPUnit según compatibilidad, rapidez, documentación y robustez.
- Implementación de más de tres ejemplos en frontend y backend.
- Análisis de resultados, ventajas, desventajas y experiencia de uso.

2. Herramientas investigadas

2.1 Frontend JavaScript

Herramienta	Fortalezas	Limitaciones	Adecuación
Vitest	ESM, mocks, modo watch, API familiar y ejecución paralela.	El DOM requiere jsdom o navegador; ecosistema más joven que Jest.	Muy alta
Jest	Ecosistema maduro, amplia documentación, mocks y snapshots.	Puede requerir más configuración para ESM y flujos modernos.	Alta

Mocha + Chai	Flexible y modular.	Exige escoger y configurar varias piezas por separado.	Media
Playwright/Cypress	Prueban flujos en navegadores reales.	Mayor costo y tiempo; su foco principal es E2E, no la unidad.	Complementaria

Testing Library se utiliza junto con Vitest para interactuar con formularios como lo haría una persona usuaria. jsdom aporta un DOM simulado y permite probar la interfaz rápidamente sin abrir un navegador.

2.2 Backend PHP/Laravel

Herramienta	Fortalezas	Limitaciones	Adecuación
PHPUnit	Estándar de facto en PHP; aserciones, data providers, mocks y cobertura.	Puede ser verboso en suites extensas.	Muy alta
Pest	Sintaxis concisa y compatible con PHPUnit.	Añade una capa y convenciones adicionales.	Alta
Codeception	Une pruebas unitarias, funcionales y de aceptación.	Configuración y alcance mayores de lo necesario.	Media
Laravel Dusk	Automatización de navegador para aplicaciones Laravel.	Más lento; adecuado para E2E, no para lógica aislada.	Complementaria

3. Herramientas seleccionadas y justificación

3.1 Vitest + jsdom + Testing Library

- Compatibilidad directa con módulos ES usados por el frontend y configuración sencilla mediante vitest.config.js.
- Mocks integrados con vi.fn() y vi.stubGlobal(), útiles para aislar fetch, navegación y almacenamiento de sesión.
- Ejecución rápida y modo watch, que vuelve a lanzar las pruebas al modificar código.
- API compatible con el estilo de Jest, lo que reduce la curva de aprendizaje.
- Testing Library favorece pruebas centradas en comportamiento observable y no en detalles internos.

3.2 PHPUnit integrado con Laravel

- Ya forma parte del ecosistema Laravel y se ejecuta con php artisan test.
- Incluye clientes de prueba HTTP, aserciones JSON y aserciones sobre la base de datos.
- RefreshDatabase y SQLite en memoria aíslan cada caso y evitan contaminar datos reales.
- Sanctum::actingAs permite comprobar autenticación y roles con bajo costo de configuración.
- Los data providers reducen duplicación cuando una regla tiene varias entradas y salidas.

Decisión

Vitest y PHPUnit ofrecen el mejor equilibrio para la arquitectura actual: ejecución rápida, integración natural con el código existente, buena documentación y suficiente capacidad para crecer hacia cobertura y CI.

4. Implementación en el proyecto

4.1 Configuración del frontend

```
// frontend/vitest.config.js
export default defineConfig({
  test: { environment: "jsdom", setupFiles: ["/tests/setup.js"],
    clearMocks: true, restoreMocks: true }
})
```

El comando `npm.cmd test` ejecuta vitest run una sola vez. `npm.cmd run test:watch` mantiene la suite activa durante el desarrollo.

4.2 Configuración del backend

```
<!-- backend/phpunit.xml -->
<testsuite name="Unit"><directory>tests/Unit</directory></testsuite>
<testsuite name="Feature"><directory>tests/Feature</directory></testsuite>
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

El comando `php artisan test` ejecuta las suites Unit y Feature usando una base de datos SQLite en memoria.

5. Ejemplos automatizados del frontend

5.1 Gestión de sesión por rol

Comprueba guardar, recuperar, restringir por rol y eliminar una sesión.

```
window.AuthSession.saveSession("abc123", user)
expect(window.AuthSession.getToken(["profesional"])).toBe("abc123")
expect(window.AuthSession.getUser(["profesional"])).toEqual(user)
```

Evidencia: tests/unit/auth-session.test.js - 3 pruebas

5.2 Construcción de URL y encabezados

Valida rutas relativas, URLs absolutas y el encabezado Bearer.

```
expect(CuidadoApi.buildUrl("/login"))
  .toBe("https://api.example.test/api/login")
expect(headers.Authorization).toBe("Bearer token-prueba")
```

Evidencia: tests/unit/api-config.test.js - 4 pruebas

5.3 Inicio de sesión

Simula el formulario, aísla la API y confirma mensaje, sesión y redirección según rol.

```
fetchJson.mockResolvedValue({ token: "token", user: { role: "admin" } })
fireEvent.click(getByRole(document.body, "button", { name: "Ingresar" }))
expect(saveSession).toHaveBeenCalled()
```

Evidencia: tests/dom/login.test.js - 4 pruebas

5.4 Registro de usuario

Comprueba campos obligatorios, payload enviado, respuesta exitosa y error del backend.

```
fireEvent.submit(document.querySelector("form"))
expect(fetchJson).toHaveBeenCalledTimes(1)
expect(JSON.parse(options.body)).toMatchObject({ role: "familiar" })
```

Evidencia: tests/dom/register.test.js - 3 pruebas

6. Ejemplos automatizados del backend

6.1 Estado del inventario de medicamentos

Un data provider cubre medicamento vencido, próximo a vencer, inventario bajo y disponible.

```
#[DataProvider('inventoryCases')]
$this->assertSame($expectedKey,
    $medication->inventoryStatus($today)['key']);
```

Evidencia: MedicationInventoryStatusTest.php - 4 conjuntos de datos

6.2 Autenticación y tokens

Verifica login exitoso, credenciales incorrectas, cuenta pendiente, perfil y revocación de tokens.

```
$this->postJson('/api/login', $credentials)
->assertOk()->assertJsonStructure(['token', 'user']);
$this->assertDatabaseCount('personal_access_tokens', 1);
```

Evidencia: AuthenticationTest.php - 8 pruebas

6.3 Rutinas del adulto mayor

Cubre CRUD, horarios inválidos, actividades completadas y restricciones por asignación.

```
$this->postJson('/api/rutinas', $payload)
->assertCreated()->assertJsonPath('rutina.horario', '08:00');
$this->assertDatabaseHas('rutinas', ['nombre' => 'Rutina matutina']);
```

Evidencia: RutinaEndpointsTest.php - 14 pruebas

6.4 Administración de usuarios

Valida autorización, creación, edición, aprobación, eliminación y rechazo de solicitudes.

```
Sanctum::actingAs($admin);
$this->patchJson("/api/admin/users/{userId}/approve")
->assertOk()->assertJsonPath('user.is_approved', true);
```

Evidencia: AdminUserManagementTest.php - 5 pruebas

7. Resultados de ejecución

Suite	Comando	Archivos	Pruebas/aserciones	Estado	Duración
Frontend	npm.cmd test	4	14 pruebas	PASS	1.45 s
Backend	php artisan test	17	88 pruebas / 405 aserciones	PASS	1.67 s

```
FRONTEND
Test Files 4 passed (4)
Tests      14 passed (14)
Duration   1.45s

BACKEND
Tests:     88 passed (405 assertions)
Duration:  1.67s
```

La evidencia demuestra que las dependencias están instaladas, las configuraciones son válidas y todos los escenarios implementados se ejecutan de forma repetible. No se observaron fallos en las dos corridas reportadas.

8. Ventajas, desventajas y experiencia

Aspecto	Vitest	PHPUnit/Laravel
Ventajas	Rápido, mocks integrados, DOM simulado, modo watch.	HTTP/JSON/BD integrados, aislamiento, soporte nativo de Laravel.
Desventajas	jsdom no reemplaza por completo un navegador real.	Las pruebas Feature pueden crecer en costo y tiempo.
Escritura	beforeEach y mocks facilitan escenarios de interfaz.	Factories y RefreshDatabase simplifican estados reproducibles.
Mantenimiento	Selectores accesibles y contratos públicos reducen fragilidad.	Conviene reutilizar factories y evitar fixtures excesivos.

La ejecución fue eficiente: 102 pruebas finalizaron en aproximadamente 3.12 segundos sumando ambas corridas observadas. El mayor esfuerzo no está en ejecutar las herramientas, sino en identificar escenarios, preparar datos y definir aserciones que representen reglas reales del sistema.

9. Conclusiones

1. La automatización permitió verificar rápidamente reglas críticas de autenticación, autorización, inventario, rutinas y formularios.
2. Vitest se ajusta bien al frontend por su soporte para ES Modules, mocks y DOM simulado; PHPUnit se ajusta al backend por su integración profunda con Laravel.
3. La combinación de pruebas unitarias y Feature ofrece mayor confianza que cualquiera de las dos categorías por separado.
4. Los resultados obtenidos - 102 pruebas aprobadas y cero fallos - establecen una línea base reproducible para detectar regresiones.
5. Como siguiente paso, se recomienda agregar cobertura de código, integrar las suites en CI y complementar los flujos principales con Playwright o Laravel Dusk en un navegador real.

10. Guion sugerido para el video

La entrega solicita un video que evidencie la implementación. El equipo puede grabar una demostración de 4 a 6 minutos con esta secuencia:

- Mostrar la estructura frontend/tests y backend/tests.
- Abrir un ejemplo de Vitest y explicar Arrange, Act y Assert.
- Ejecutar `npm.cmd test` y mostrar 14 pruebas aprobadas.
- Abrir un ejemplo de PHPUnit y explicar datos, petición y aserciones.
- Ejecutar `php artisan test` y mostrar 88 pruebas y 405 aserciones aprobadas.
- Cerrar con ventajas, limitaciones y próximos pasos.

Enlace del video

[PEGAR AQUÍ EL ENLACE AL VIDEO ANTES DE ENTREGAR EL PDF]

Referencias

- Vitest. Why Vitest. <https://vitest.dev/guide/why.html>
- Vitest. Getting Started. <https://vitest.dev/guide/>
- Vitest. Features. <https://vitest.dev/guide/features.html>
- PHPUnit. Documentation. <https://phpunit.de/documentation.html>
- PHPUnit. Manual. <https://docs.phpunit.de/>
- Jest. Getting Started. <https://jestjs.io/docs/getting-started>
- Playwright. Test documentation. <https://playwright.dev/docs/test-intro>
- Laravel. HTTP Tests. <https://laravel.com/docs/11.x/http-tests>
- Testing Library. Guiding Principles. <https://testing-library.com/docs/guiding-principles>